

Python Topic 0 — Installation & Setup

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • Latest stable: Python 3.14.x

Layer 1 — New to Python

Python is a program you install on your computer. Once installed, you can run `.py` files (your code) and type commands into an interactive prompt.

Install it

Go to python.org/downloads, download the latest version (3.14.x), and run the installer.

⚠ Windows — most important step: tick "Add Python to PATH" on the first installer screen. Skip it and the terminal won't find Python.

Check it worked

Open a terminal and type:

```
python3 --version
```

You should see something like `Python 3.14.6`. On macOS/Linux, `python3` is the reliable command (see Layer 3 for why). Windows users can use `python` or the `py` launcher.

Run your first code

The interactive shell — type `python3` and press Enter:

```
>>> print("Hello, Python!")
Hello, Python!
>>> 2 + 3
5
>>> exit()
```

Or save a file `hello.py` and run it:

```
python3 hello.py
```

Layer 2 — Intermediate (real setup you'll use)

Coming from Node.js, here's the mental mapping:

Node.js world	Python world
<code>node</code>	<code>python3</code>
<code>npm</code> / <code>yarn</code>	<code>pip3</code>
<code>package.json</code>	<code>pyproject.toml</code> / <code>requirements.txt</code>
<code>node_modules/</code>	virtual environment (<code>venv</code>)
<code>nvm</code> (version manager)	<code>pyenv</code> or <code>uv</code>
<code>npx</code>	<code>pipx</code> / <code>uv run</code>

The critical concept: virtual environments

In Node, dependencies install locally into `node_modules/` by default. In Python, `pip3 install` installs **globally** unless you create a virtual environment first. A venv is an isolated per-project folder for packages.

```
# Create a virtual environment (creates a .venv folder)
python3 -m venv .venv

# Activate it
# macOS / Linux:
source .venv/bin/activate
# Windows (PowerShell):
.venv\Scripts\Activate.ps1

# Inside the venv, plain python/pip now point to Python 3 correctly
pip install fastapi

# Save your dependencies (like package.json)
pip freeze > requirements.txt

# Later, on another machine, reinstall them
pip install -r requirements.txt

# Leave the venv
deactivate
```

Note: Once activated, your prompt shows `(.venv)` at the start. **Before** activating you use `python3` / `pip3` ; **once activated**, plain `python` / `pip` work and point to the venv.

Editor

VS Code (which you already use with Cursor) + the official **Python extension** by Microsoft gives you IntelliSense, debugging, and venv auto-detection. Select the interpreter with `Cmd/Ctrl+Shift+P` → "Python: Select Interpreter" → pick your `.venv`.

Layer 3 — Expert (modern tooling & gotchas)

1. Why `python3` and not `python` ?

On macOS/Linux, `python` historically pointed to the old Python 2, so these systems keep `python3` as the explicit, safe name for Python 3 — and often don't create a plain `python` command at all. That's why `python3 --version` works while `python` may say "command not found." Both run the same interpreter; `python3` is just guaranteed. On Windows, the `py` launcher picks versions: `py -3.14 script.py`. Inside an activated venv, plain `python` points to the venv's Python everywhere.

2. Never install globally / never `sudo pip3`

 **Avoid:** Installing packages system-wide pollutes the OS Python and causes version conflicts. Modern Python (PEP 668) even blocks this with an "externally-managed-environment" error. Always work inside a venv. (This is why some sandboxes use `pip install --break-system-packages` — an override you should avoid on your real machine.)

3. `uv` — the modern all-in-one (recommended in 2026)

`uv` (by Astral, makers of the Ruff linter) is a Rust-based tool that replaces `pip`, `venv`, `pyenv`, and `pipx` — and it's dramatically faster (10–100×). From the JS world, think of it as the "pnpm/bun moment" for Python.

```
# Install uv (macOS/Linux)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Start a new project (creates pyproject.toml + .venv automatically)
uv init myproject
cd myproject

# Add a dependency (resolves + installs + updates lockfile)
uv add fastapi

# Run code inside the managed env – no manual activation needed
uv run main.py

# Install a specific Python version itself
uv python install 3.14
```

✓ **Tip:** `uv` gives you a real lockfile (`uv.lock`) like `package-lock.json`, which plain `pip3` + `requirements.txt` doesn't do well. For your AI/FastAPI work, start with `uv` directly.

4. `pyproject.toml` is the modern standard

The old style was a loose `requirements.txt`. The modern, standardized project descriptor is `pyproject.toml` — the direct analogue to `package.json` (metadata + dependencies + tool config in one file).

Interview Q&A Cards

Q: Why does `python3` work but `python` sometimes doesn't?

A: On macOS/Linux, `python` historically meant Python 2, so distros expose Python 3 as `python3` and often omit a plain `python`. Both run the same interpreter. Inside a venv, plain `python` resolves to the venv's version.

Q: What is a virtual environment and why use one?

A: An isolated per-project folder for dependencies. Without it, `pip` installs globally and projects clash. It's Python's answer to `node_modules/`, but you must create and activate it yourself.

Q: `pip + venv` vs `uv` — what's the difference?

A: `python3 -m venv + pip` is the classic built-in combo. `uv` is a modern Rust-based replacement that's far faster and also manages Python versions and a real lockfile (`uv.lock`), similar to `pnpm / bun`.

 **Memory hook:** `python3` is the safe name outside a venv; inside a venv, plain `python` just works.

✓ **One-liner for interviews:** "Every Python project should run in an isolated virtual environment to avoid dependency conflicts. `python3 -m venv + pip` is the classic combo; `uv` is the modern high-speed replacement that also manages Python versions and lockfiles."